DSP in VLSI
Homework 1 - Sorter

Jen-Tse Chen

March 2026

1. First, generate several random sequences with 32 elements $[x_1, x_2, \ldots, x_{32}]$ in a block between -256 and 255 as your test inputs and prepare as the testbench. (10%)

   A Python script was used to generate random sequences of 32 elements with values ranging from -256 to 255. These sequences are stored in `input.txt`.

   The testbench reads this file to provide a continuous stream of input blocks for the simulation. For the corresponding signal transitions, please refer to the waveforms provided in Figure 6.

2. Please use Verilog to implement `Sort8` with bitonic sorter. Feed the 8 inputs $[x_1, x_2, \ldots, x_8]$ simultaneously in one clock cycle. Observe the outputs to realize sorting among eight elements in one group. Use $[x_1, x_2, \ldots, x_8]$ as the input and check for the correctness of the function "`Sort8`", which can generate sorted output from maximum to minimum. Indicate this function output in the timing diagram of your behavior simulation results. (If the data value inside the timing diagram can not be clearly seen, it will not be regarded as correct answers.) (10%)

   The functional correctness of the `Sort8` module was verified via behavioral simulation. Eight signed integers were fed simultaneously into the bitonic sorter within a single clock cycle. The simulation output (shown in Figure 1) matches the Golden Model generated by the Python script: `[254, 220, 126, 109, 66, -26, -158, -239]`.

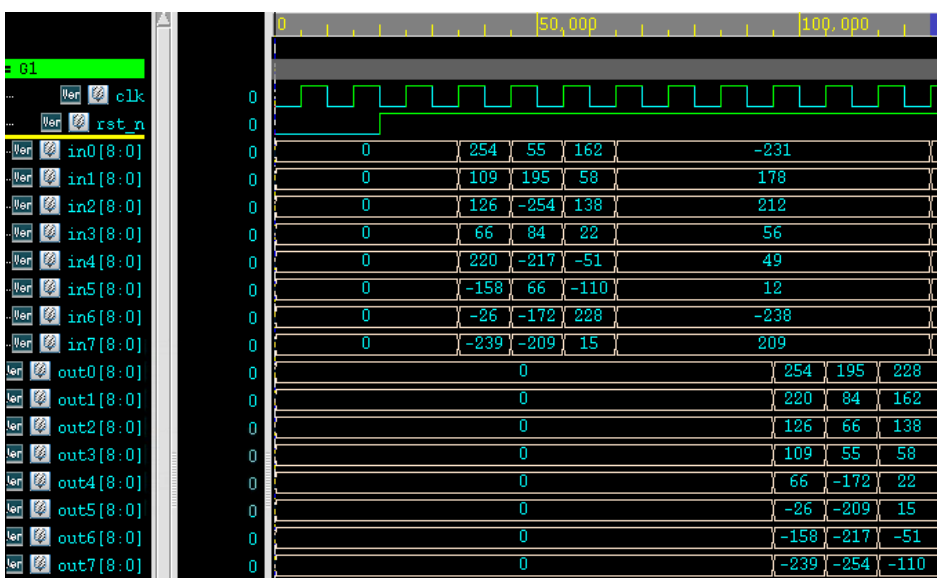

Figure 1: Verification of module `Sort8`

3. Construct the module (`SelectTopK`) of merger sort to select top 4 among 32 input elements ($[x_1, x_2, x_3, \ldots, x_{32}]$) under your control. Use a comparator tree to receive the sorted results from each group and then activate your comparator tree to select top 4 values. Maybe you need two sets of `Reg8` buffers, one for saving the outputs of the next block from `Sort8`, and the other for sending the contents of the current block to the comparator tree. Draw the block diagram of your own implementation and calculate the number of adders/subtractors/comparators in your block diagram. (20%)
Show the timing diagram of your behavior simulation results (10%)
Show the hardware output is correct by comparing it to your Matlab results. (10%)

The design implements a hierarchical selection network to extract the top-4 elements from 32 inputs. Based on the architectural diagrams (Figures 2 through 5), the hardware resource consumption is as follows:

(a) Module `Sort8` utilizes 24 9-bit comparators (based on the standard $N \log^2(N)$ stages for a bitonic network).

(b) Module of comparator tree used to merge and select the top-4 values consists of 3 9-bit comparators.

(c) The complete module `SelectTopK` utilizes 27 9-bit comparators.

The behavioral simulation results, illustrated in Figure 6, confirm the module's ability to process 32 elements across four distinct blocks. The hardware output was verified against the Golden Model generated by the Python script:

```
Block 0: [254 220 174 164]
Block 1: [248 225 195 189]
Block 2: [236 235 228 203]
Block 3: [248 233 219 217]
```

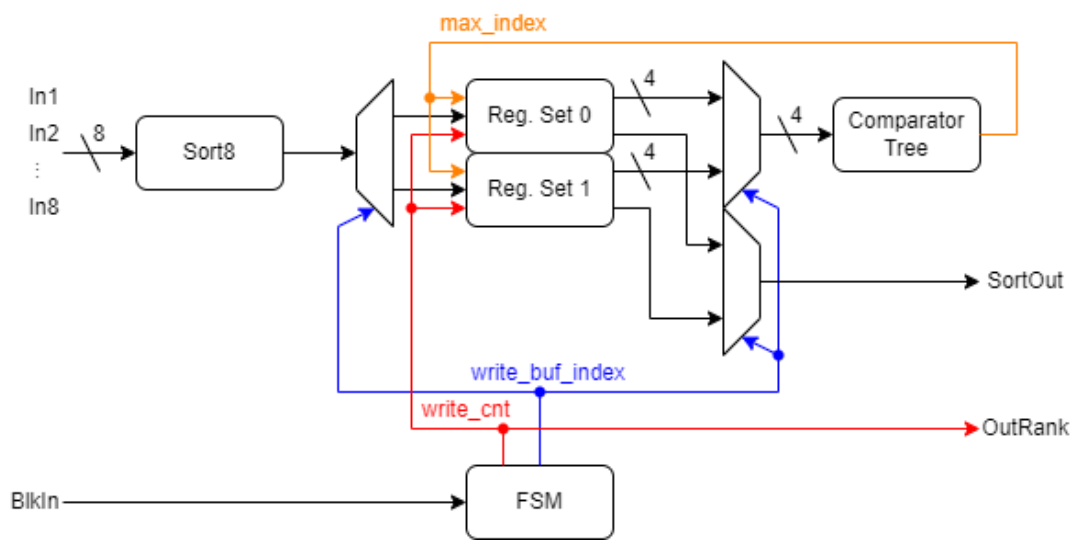

Figure 2: Design of module SelectTopK

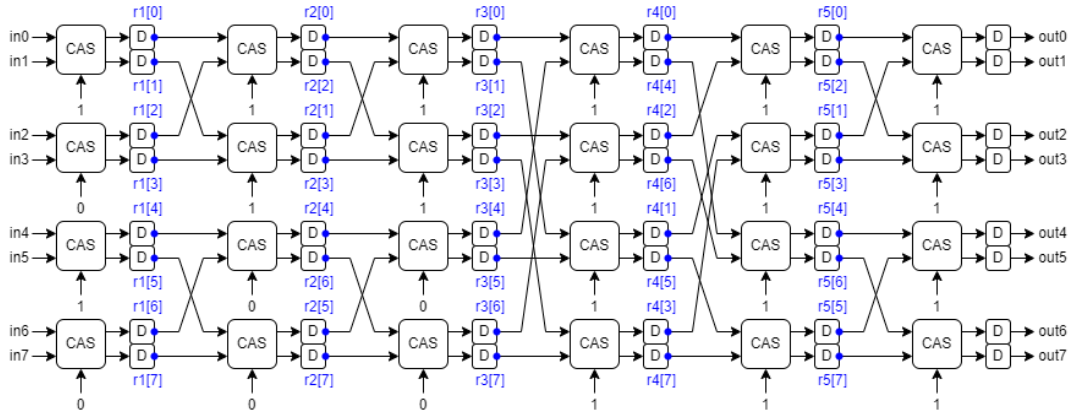

Figure 3: Design of module `Sort8` (CAS is Compare And Swap, refer to Figure 4 for details)

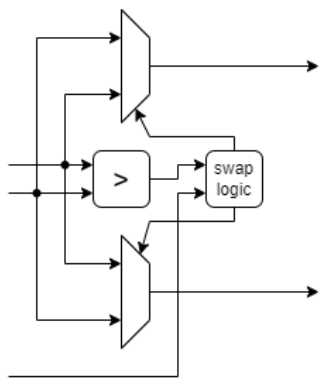

Figure 4: Design of module of compare and swap

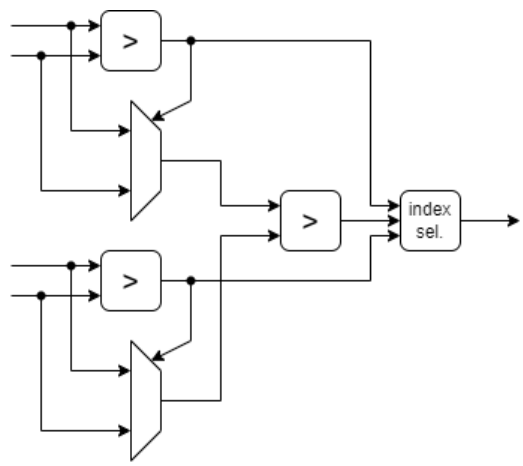

Figure 5: Design of module of comparator tree

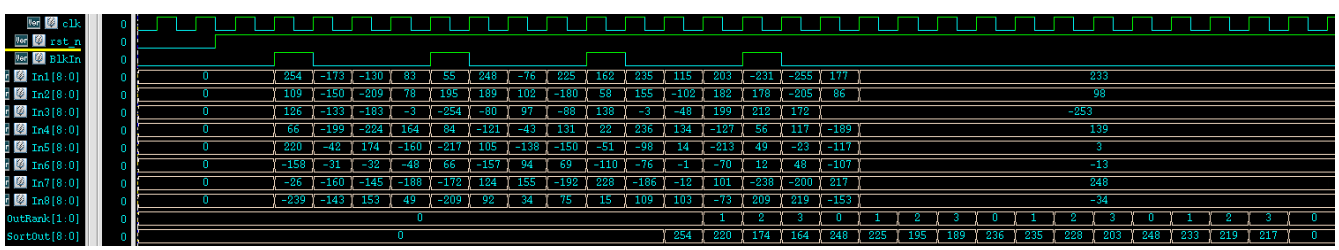

Figure 6: Verification waveform (input and output) of module `SelectTopK`

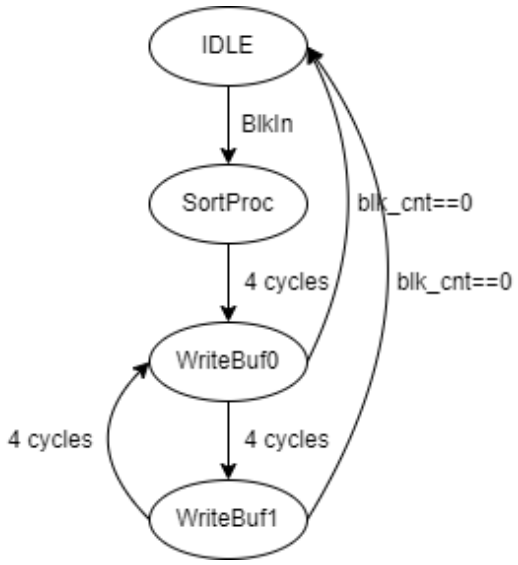

Figure 7: Finite state machine of module `SelectTopK`

4. Show that your design can generate required top-4 output in descending order every 4 clock cycles with proper input signal "`BlkIn`" and output indicator "`OutRank`". Please feed 4 blocks and check the outputs of four blocks. If it is too long, please cut it down into several segments to make the numerical expressions in the timing diagram clear. If the numerical expressions are hard to distinguish, correct evaluation may not be given. (30%)

   As shown in Figure 6, the design successfully generates the top-4 output in descending order every 4 clock cycles. The `BlkIn` signal effectively triggers block processing, while `OutRank` correctly identifies the sorted position of each element.

   To sustain continuous streaming input, the design implements:

   (a) Ping-Pong Buffering: A dual-buffer scheme (Fig. 2) allows simultaneous sorting of new data while selecting the Top-4 from the previous set.

   (b) FSM Control: A Finite State Machine (Fig. 7) orchestrates the buffer switching and signal timing, ensuring a seamless transition across the 4-block sequence. Upon receiving the `BlkIn` signal, the `blk_cnt` register increments. Conversely, once a block output operation completes, `blk_cnt` decrements. The ping-pong buffering state machine transitions back to IDLE only when `blk_cnt` reaches zero.

5. Please use Matlab command "sort" to verify your results of your randomly-generated sequence and compare to the Verilog simulation results (10%).

   The functional correctness of the design is verified against the randomly generated input and the Golden Model described in Q3. Both methods confirm that the hardware output is perfectly aligned with the expected sorted results.

6. Synthesis your design in Q4. Show the number of adders/subtractors/ comparators in your design. Sum them up together to see if it matches with your block diagram. (10%) (Because you may use counters to control your circuits, we will not ask that the two values should be exactly the same. However, there should not be a large discrepancy.)

   The design was synthesized at a clock frequency of 200 MHz. The resulting timing and area reports are provided in Figures 8, 9, and 10, and the resource hierarchy report is shown in Figure 11.

   The architectural design specifies 27 9-bit comparators as detailed in the resource hierarchy (Figure 11): 24 within the `Sort8` module (one per `compare_swap` instance across the bitonic network) and 3 within the `max_index_finder` comparator tree.

```
Startpoint: fsm0/current_state_reg[1]
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: SortOut[1] (output port clocked by clk)
Path Group: clk
Path Type: max

Des/Clust/Port     Wire Load Model       Library
------------------------------------------------
SelectTopK         ZeroWireload          N16ADFP_StdCellff0p88v125c_ccs

Point                                                   Incr       Path
-----------------------------------------------------------------------
clock clk (rise edge)                                   0.00       0.00
clock network delay (ideal)                             0.50       0.50
fsm0/current_state_reg[1]/CP (DFCNQD1BWP20P90)          0.00 #     0.50 r
fsm0/current_state_reg[1]/Q (DFCNQD1BWP20P90)           0.04       0.54 r
U4335/ZN (INVD1BWP20P90)                                0.01       0.55 f
U4171/ZN (NR2D1BWP20P90)                                0.05       0.61 r
U4104/Z (CKBD1BWP20P90)                                 0.02       0.63 r
U3749/ZN (INVD1BWP20P90)                                0.04       0.67 f
U3658/Z (AN4D1BWP20P90)                                 0.02       0.69 f
U3415/ZN (AOI22D1BWP20P90)                              0.01       0.70 r
U3201/ZN (INVD1BWP20P90)                                0.01       0.71 f
U2913/ZN (MAOI222D1BWP20P90)                            0.01       0.73 r
U2889/ZN (MAOI222D1BWP20P90)                            0.01       0.74 f
U2818/ZN (MAOI222D1BWP20P90)                            0.01       0.76 r
U2595/ZN (MAOI222D1BWP20P90)                            0.01       0.77 f
U2472/ZN (MAOI222D1BWP20P90)                            0.01       0.78 r
U2467/ZN (MAOI222D1BWP20P90)                            0.01       0.80 f
U2460/ZN (MAOI222D1BWP20P90)                            0.01       0.81 r
U2457/ZN (MAOI222D1BWP20P90)                            0.07       0.88 f
U2445/ZN (MUX2ND1BWP20P90)                              0.04       0.92 r
U2430/ZN (OAI211D1BWP20P90)                             0.02       0.94 f
U2424/ZN (MAOI222D1BWP20P90)                            0.02       0.95 r
U2422/ZN (NR2D1BWP20P90)                                0.01       0.96 f
U2420/ZN (OAI22D1BWP20P90)                              0.01       0.98 r
```

Figure 8: Timing report (1/2)

```
U2412/ZN (ND2D1BWP20P90)                          0.01       1.07 r
U2411/ZN (OAI21D1BWP20P90)                        0.01       1.08 f
U2410/ZN (OAI32D1BWP20P90)                        0.02       1.09 r
U2409/ZN (MAOI222D1BWP20P90)                      0.04       1.13 f
U2408/ZN (INVD1BWP20P90)                          0.02       1.15 r
U2391/ZN (NR2D1BWP20P90)                          0.03       1.18 f
U2387/ZN (AOI22D1BWP20P90)                        0.02       1.20 r
U2370/ZN (AOI21D2BWP20P90)                        0.10       1.30 f
SortOut[1] (out)                                  0.00       1.30 f
data arrival time                                            1.30

clock clk (rise edge)                             5.00       5.00
clock network delay (ideal)                       0.50       5.50
clock uncertainty                                -0.10       5.40
output external delay                            -2.50       2.90
data required time                                           2.90
-------------------------------------------------------------------
data required time                                           2.90
data arrival time                                           -1.30
-------------------------------------------------------------------
slack (MET)                                                  1.60
```

Figure 9: Timing report (2/2)

```
Library(s) Used:

    N16ADFP_StdCellff0p88v125c_ccs (File: /share1/tech/ADFP/Executab

Number of ports:                          86
Number of nets:                         3503
Number of cells:                        3428
Number of combinational cells:          2396
Number of sequential cells:             1032
Number of macros/black boxes:              0
Number of buf/inv:                       599
Number of references:                     35

Combinational area:               755.568016
Buf/Inv area:                      97.200004
Noncombinational area:           1177.079083
Macro/Black Box area:               0.000000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                 1932.647099
Total area:                 undefined
```

Figure 10: Area report

```
Resource Report for this hierarchy in file ../01_RTL/SelectTopK.sv
===============================================================================
| Cell            | Module         | Parameters | Contained Operations       |
===============================================================================
| add_x_5         | DW01_inc       | width=3    | add_80_S2 (SelectTopK.sv:80) |
| add_x_15        | DW01_inc       | width=3    | fsm0/add_31_S2 (merge_sort_fsm.sv:31) |
| add_x_17        | DW01_inc       | width=2    | fsm0/add_38 (merge_sort_fsm.sv:38) |
| add_x_19        | DW01_inc       | width=4    | fsm0/add_45_S2 (merge_sort_fsm.sv:45) |
| gt_x_28         | DW_cmp         | width=4    | fsm0/gt_79 (merge_sort_fsm.sv:79) |
| gt_x_29         | DW_cmp         | width=9    | max0/gt_17 (max_index_finder.sv:17) |
| gt_x_30         | DW_cmp         | width=9    | max0/gt_25 (max_index_finder.sv:25) |
| gt_x_31         | DW_cmp         | width=9    | max0/gt_33 (max_index_finder.sv:33) |
| gt_x_35         | DW_cmp         | width=9    | sort0/cs1_0/gt_13 (compare_swap.sv:13) |
| gt_x_38         | DW_cmp         | width=9    | sort0/cs6_3/gt_13 (compare_swap.sv:13) |
| gt_x_41         | DW_cmp         | width=9    | sort0/cs6_2/gt_13 (compare_swap.sv:13) |
| gt_x_44         | DW_cmp         | width=9    | sort0/cs6_1/gt_13 (compare_swap.sv:13) |
| gt_x_47         | DW_cmp         | width=9    | sort0/cs6_0/gt_13 (compare_swap.sv:13) |
| gt_x_50         | DW_cmp         | width=9    | sort0/cs5_3/gt_13 (compare_swap.sv:13) |
| gt_x_53         | DW_cmp         | width=9    | sort0/cs5_2/gt_13 (compare_swap.sv:13) |
| gt_x_56         | DW_cmp         | width=9    | sort0/cs5_1/gt_13 (compare_swap.sv:13) |
| gt_x_59         | DW_cmp         | width=9    | sort0/cs5_0/gt_13 (compare_swap.sv:13) |
| gt_x_62         | DW_cmp         | width=9    | sort0/cs4_3/gt_13 (compare_swap.sv:13) |
| gt_x_65         | DW_cmp         | width=9    | sort0/cs4_2/gt_13 (compare_swap.sv:13) |
| gt_x_68         | DW_cmp         | width=9    | sort0/cs4_1/gt_13 (compare_swap.sv:13) |
| gt_x_71         | DW_cmp         | width=9    | sort0/cs4_0/gt_13 (compare_swap.sv:13) |
| gt_x_74         | DW_cmp         | width=9    | sort0/cs3_3/gt_13 (compare_swap.sv:13) |
| gt_x_77         | DW_cmp         | width=9    | sort0/cs3_2/gt_13 (compare_swap.sv:13) |
| gt_x_80         | DW_cmp         | width=9    | sort0/cs3_1/gt_13 (compare_swap.sv:13) |
| gt_x_83         | DW_cmp         | width=9    | sort0/cs3_0/gt_13 (compare_swap.sv:13) |
| gt_x_86         | DW_cmp         | width=9    | sort0/cs2_3/gt_13 (compare_swap.sv:13) |
| gt_x_89         | DW_cmp         | width=9    | sort0/cs2_2/gt_13 (compare_swap.sv:13) |
| gt_x_92         | DW_cmp         | width=9    | sort0/cs2_1/gt_13 (compare_swap.sv:13) |
| gt_x_95         | DW_cmp         | width=9    | sort0/cs2_0/gt_13 (compare_swap.sv:13) |
| gt_x_98         | DW_cmp         | width=9    | sort0/cs1_3/gt_13 (compare_swap.sv:13) |
| gt_x_101        | DW_cmp         | width=9    | sort0/cs1_2/gt_13 (compare_swap.sv:13) |
| gt_x_104        | DW_cmp         | width=9    | sort0/cs1_1/gt_13 (compare_swap.sv:13) |
| sub_x_20        | DW01_dec       | width=4    | fsm0/sub_46_S2 (merge_sort_fsm.sv:46) |
      |                 |            | fsm0/sub_47_S2 (merge_sort_fsm.sv:47) |
===============================================================================
```

Figure 11: Resource hierarchy report